

Q1 ~~print~~ Hello in java

→ public class hello {

public static ~~hello~~ void main(String[]
args) {

System.out.println("hello");

}

Q2 ~~print~~ Add two numbers in java

→ public class add {

public static void main(String[] args)

{
int a=10, b=20, c;

c = a+b;

System.out.println(c);

}

}

[3] Find Factorial in java

```
→ public class Fact {  
    public static void main(String[] args)  
    {  
        int Fact = 1;  
        For(int i = 5; i > 1; i--) {  
            Fact = Fact * i;  
        }  
        System.out.println(Fact);  
    }  
}
```

[4] Add two numbers taken from a user

```
import java.util.Scanner;
```

```
public class Scanadd {  
    public static void main(String[] args)  
    {  
        int a, b, c;  
        Scanner s = new Scanner(System.in);  
        a = s.nextInt();  
        b = s.nextInt();  
        c = a + b;  
        System.out.println(c);  
    }  
}
```


Q5] Find Factorial numbers taken for a user in Java

→ import java.util. Scanner;

```
public class scanfact {  
    public static void main(String[] args)  
    {  
        int fact = 1, n;  
        Scanner S = new Scanner(System.in);  
        n = S.nextInt();  
  
        for(int i = n; i > 1; i--) {  
            fact = fact * i;  
        }  
        System.out.println(fact);  
    }  
}
```

[6] print helloworld in notepad and
Run in terminal

→ public class hello

{

public static void main(String[]
args)

{

System.out.print("helloworld");

}

}

[7]

a. java save

class b

{

public static void main(String[] args)

{

System.out.print("Janvi parusai");

}

[8] Use Scanner class and print integers, float and string value

- import java.util.Scanner;

public static void main(String[] args) {

Scanner var = new Scanner(System.in);

String s = var.nextLine();
System.out.println("The string value is:"
+ s);

int i = var.nextInt();
System.out.println("The int value is:" + i);

Float f = var.nextFloat();
System.out.println("The float value is:"
+ f);

[9] Use OF console class in NOTepad

→ public class console {

public static void main (String[] args) {

String s = System.console().readLine();

System.out.println("The string value is :
" + s);

}

}

[10] print 5 Name using command line argument

```
- public class CMD {
```

```
    public static void main(String[] args)
```

```
    {
        int n = args.length;
```

```
        int i = 0;
        while (i < n) {
```

```
            System.out.println("Java: " + args[i]);
```

```
            i = i + 1;
```

```
        }
```

```
    }
```

```
}
```

[11] create calculator using Switchcase

→

```
public static void main(String[] args) {
```

```
    Scanner in = new Scanner(System.in);
```

```
    System.out.print("Enter the value of a:");  
    int a = in.nextInt();
```

```
    System.out.print("Enter the value of b:");  
    int b = in.nextInt();
```

```
    System.out.print("Enter the operation  
                    :");
```

```
    char c = in.next().charAt(0);
```

```
    Switch(c) {
```

```
        case '+':
```

```
            System.out.print(a+b);  
            break;
```

```
        case '-':
```

```
            System.out.print(a-b);  
            break;
```

```
        case '/':
```

```
            System.out.print(a/b);  
            break;
```


[12] print Fibonacci series

→ import java.util.Scanner;

class F; {

void Fibol() {

int F1=0, F2=1, F3;

System.out.println("Enter a number
:");

Scanner in = new Scanner(System
.in);

int n = in.nextInt();

for (int i=0; i<=n; i++)

{

System.out.println(F1);

F3 = F1 + F2;

F1 = F2;

F2 = F3;

}

}

}

public class Fibona {

public static void main(String[] args)

{

F1 obj = new F1();

[13] program to check prime number

```
import java.util. scanner;
```

```
class Prime {
```

```
    boolean (int n) {
        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
```

```
                return false;
```

```
        } }
```

```
    public class is return true;
```

```
}
```

```
public class isprime {
```

```
    public static void main (String args) {
```

```
        Scanner in = new Scanner (System.in);
```

```
        System.out.print ("Enter the number:");
```

```
        int n = in.nextInt();
```

```
        Prime obj = new Prime();
```

```
        boolean ans = obj.isprime(n);
        if (ans) {
```

```
            System.out.print ("is prime");
```

```
        }
        else {
```

```
            System.out.print ("Not prime");
```

```
        }
```

```
}
```

```
}
```


Q.1] print person is 18+ or not by taking age & country as indicator.

```

→ import java.util.Scanner;

public class age {

    public static void main(String args[]) {
        Scanner S = new Scanner(System.in);

        int n = S.nextInt();

        System.out.println("Enter age: ");
        int n = S.nextInt();

        System.out.println("Enter country: ");
        boolean indian = S.nextBoolean();

        if (n > 18 && indian == true) {
            System.out.println("18+ vote in election");
        }
        else {
            System.out.println("not age");
        }
    }
}

```

[15] create array and take from user and print the values.

→ import java.util.*;

public class takevalueofArray {

public static void main(String args[]) {

Scanner s = new Scanner(System.in);

System.out.println("Enter size of array:");

int n = s.nextInt();

String[] a = new String[n];

System.out.println("Enter a element of array:");

for(i=0; i<a.length; i++) {

a[i] = s.nextLine();

}

for(i=0; i<a.length; i++) {

System.out.println(a[i]);

}

}

[16] Create Array and print it's value

```
public class staticArr {  
    public static void main(String args[])  
    {  
        int[] arr = {10, 20, 30, 40, 50};  
        for(int i=0; i<arr.length; i++) {  
            System.out.println(arr[i]);  
        }  
    }  
}
```

[17] print string array

```
public class StringArray {  
    public static void main(String args[]) {  
        String[] a = {"IBM", "RBI", "SBI", "BOB",  
                       "ABC"};  
        for(int i=0; i<a.length; i++) {  
            System.out.println(a[i]);  
        }  
    }  
}
```

[18] using logical operators.

```
→ public class logical {  
    public static void main(String args[]) {  
        int x=5, y=7, z=3;  
        if ((x>3 && y<10) || (z==3 && x<=5)) {  
            System.out.println("Both condition are  
                                true");  
        }  
        boolean a = true;  
        if (!a) {  
            System.out.println("The value OF x is  
                                False");  
        }  
        else {  
            System.out.println("The value OF x is  
                                true");  
        }  
    }  
}
```


[19] create a program of Arithmetic operator

```
→ import java.util.Scanner;

public class Arithmetic {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("enter a First number");
        double num1 = sc.nextDouble();

        System.out.println("enter a second number");
        double num2 = sc.nextDouble();

        double sum = num1 + num2;
        double difference = num1 - num2;
        double product = num1 * num2;
        double quotient = num1 / num2;

        System.out.println(+sum);
        System.out.println(+difference);
        System.out.println(+product);
        System.out.println(+quotient);
    }
}
```

[20] Relation Operator

```
→ import java.util.Scanner;
```

```
public class relation {
```

```
    public static void main(String[] args) {
```

```
        Scanner scan = new Scanner(System.in);  
        System.out.println("enter a first number");  
        int num1 = scan.nextInt();
```

```
        System.out.println("enter the second number");  
        int num2 = scan.nextInt();
```

```
        System.out.println(+num1 > num2);  
        System.out.println(+num1 < num2);
```

```
        System.out.println(+num1 >= num2);
```

```
        System.out.println(+num1 <= num2);
```

```
        System.out.println(+num1 == num2);
```

```
        System.out.println(+num1 != num2);
```

```
    }
```

```
}
```


[21] Logical operator

→ public class logical {

public static void main(String[] args) {

boolean a = true;

boolean b = false;

System.out.println("a:" + a);

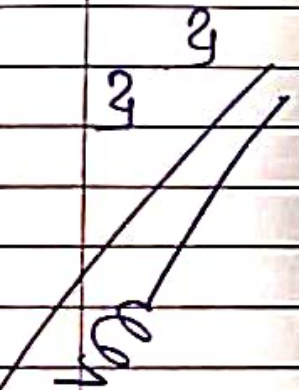
System.out.println("b:" + b);

System.out.println("a & b:" + (a & b));

System.out.println("a || b:" + (a || b));

System.out.println("!a:" + !a);

System.out.println("!b:" + !b);



[22] ^{Bitwise} ~~Ternary~~ operator

```
public class bitwise {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int a = s.nextInt();
        System.out.println("enter value OF a: " + a);
        int b = s.nextInt();
        System.out.println("enter value OF b: " + b);
        System.out.print(a & b);
        System.out.print(a | b);
        System.out.print(a ^ b);
    }
}
```


23] Ternary operator

```
public class ternary {  
    public static void main (String[] args) {  
        int a = 10;  
        int b;  
        b = (a == 1) ? 61 : 90;  
        System.out.println(b);  
    }  
}
```

(28) Break, continue, return

```
→ import java.util Scanner;
public class addition {
    public static void main (String arg []) {
        Scanner S = new Scanner (System.in);
        int S = 0;
        while (true)
            System.out.print ("Enter a value?");
            int num = S.nextInt();

            if (num < 0) {
                System.out.print ("Negative");
                break;
            }
            if (num == 0) {
                System.out.print ("Zero is ignored");
                continue;
            }
            S += num;
        }
        System.out.print (S);
        return;
    }
}
```


[25] LITERALS

- Integer

program 1 : Octal

```
class main {  
    public static void main(String[] args) {  
        int a = 20;  
        System.out.println(a);  
    }  
}
```

program 2 : Binary

```
class main {
```

```
    public static void main(String[] args)  
    {  
        int a = 0b1111;  
        System.out.println(a);  
    }  
}
```

program 3 : hexadecimal

```
class Main {
```

```
    public static void main(String[] args)
```

```
    {  
        int a = 0xA;  
        System.out.println(a);
```

```
    }
```

[2] Floating point literals

→ class Main {

```
    public static void main(String[] args)
```

```
    {  
        Float a = 12.5F;
```

```
        System.out.println(a);
```

```
    }
```

```
}
```


program 2 : double

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        double a = 253636;
```

```
        System.out.println(a);
```

```
    }
```

```
}
```

[2nd] characters literals

program : 1

```
class Main {
```

```
    public static void main(String args)
```

```
    {
```

```
        char c = 'a';
```

```
        System.out.println(c);
```

```
    }
```

```
}
```

program 2 :

```
class Main {
```

```
    public static void main(String[] args)
```

```
    {
```

```
        char a = '10061';
```

```
        System.out.println(a);
```

```
    }
```

```
}
```

program 3 : (ln)

```
class Main {
```

```
    public static void main(String[] args)
```

```
    {
```

```
        char a = '\n';
```

```
        System.out.println(a);
```

```
    }
```

```
}
```


[27] Runs all String Function

(1) String append():

```
StringBuffer sb = new StringBuffer("Hello");  
sb.append("world");  
System.out.println(sb);
```

(2) compare():

```
StringBuffer sb1 = new StringBuffer("Hello");  
StringBuffer sb2 = new StringBuffer("Hello");  
System.out.println(sb1.toString().  
compareTO(sb2.toString()));
```

(3) equals():

```
StringBuffer sb1 = new StringBuffer("Hello");  
StringBuffer sb2 = new StringBuffer("world");  
System.out.println(sb1.toString().equals(  
sb2.toString()));
```

(4) equalsIgnoreCase():

```
String sb1 = new String("Hello");  
String sb2 = new String("Hello");  
System.out.println(sb1.equalsIgnoreCase(  
sb2));
```

(5) join():

```
String result = String.join(", ""Hello",  
    System.out.print(result); "world");
```

(6) start and with():

```
String str = "Java is Coding language";  
System.out.print(str.startsWith("Java"));
```

(7) Replace():

```
String a = "Hello";  
System.out.print(a.replace(a, "Java"));
```

(8) position():

```
StringBuffer sb = new StringBuffer("Java");  
System.out.print(sb.charAt(1));
```

(9) Reverse():

```
StringBuffer sb = new StringBuffer  
    ("Java");  
System.out.print(sb.reverse());  
System.out.print(sb.reverse());
```


(10) Insert():

```
StringBuffer sb = new StringBuffer("Java");
```

```
String sb.insert(4, "script");  
System.out.println(sb);
```

(11) delete()

```
StringBuffer sb = new StringBuffer("Java  
world");
```

```
System.out.println(sb.delete(5, 11));
```

(12) EnsureCapacity()

```
String str1 = "Hello";
```

```
String str2 = ""
```

```
System.out.println(str1.concat(str2));
```

(13) checkCapacity()

```
StringBuffer sb = new StringBuffer();  
System.out.println(sb.capacity());
```

(28) Add of 100 number but not add number in divided by 2 & 6 :

```
→ public class exam {  
    public static void main (String args[]) {  
        int i;  
        int sum = 0;  
        for (i = 1; i <= 100; i++)  
        {  
            if (i % 2 == 0 && i % 6 == 0)  
            {  
                sum = i + sum;  
            }  
        }  
    }  
}
```


[29] Print table in 5 to using a loop's in include a switch case

```
→ import java.util.Scanner;  
public class exam {  
    public static void main (String args[]) {  
        int num = 5;  
        Scanner s = new Scanner (System.in);  
        System.out.println ("Enter a choice");  
        int n = s.nextInt();  
        int i = 1;  
        Switch (n) {  
            case 1 : for (i=1; i<=10; i++) {  
                System.out.print (num + "*" + i + " + i * 5);  
                break; }  
            case 2 : while (i<=10) {  
                System.out.print (num + "*" + i + " + i * 5);  
                i = i + 1;  
                break;  
            }  
            case 3 : do {  
                System.out.print (num + "*" + i + " + i * 5);  
                i = i + 1;  
            }  
            while (i<=10);  
        }  
    }  
}
```

(30)

Wrapper class

→

```
import java.util. ArrayList;
```

```
class main {
```

```
    public static void main (String args[])
```

```
    {
```

```
        ArrayList<Integer> a = new ArrayList<Integer>();
```

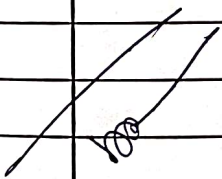
```
        a.add (1) ;
```

```
        a.add (2) ;
```

```
        a.add (3) ;
```

```
        System.out.print(a);
```

```
    }
```



(31) Factorial using 2 class with return.

```
→ class fact {  
    int fact = 1;  
    int fact()  
    {  
        for (int i = 1; i <= 5; i++)  
        {  
            fact = fact * i;  
        }  
        return fact;  
    }  
}  
  
public class factclass {  
    public static void main (String args[])  
    {  
        fact obj = new fact();  
        int x = (obj.fact());  
        System.out.println(x);  
    }  
}
```

m

(32) Method call with two class

→

```
class a {  
    int a=1, b=2;  
    void sum () {  
        System.out.println(a+b);  
    }  
    void mul () {  
        System.out.println(a+b);  
    }  
    void s () {  
        System.out.println(a-b);  
    }  
}
```

```
public class b {  
    public static void main (String [] args) {  
        a ob = new a();  
        ob.mul();  
        ob.s();  
    }  
}
```

ms

(33) Write down Java Program to use for class & object.

→ class Box {

double w = 1;

double h = 2;

double d = 3;

double volume() {

return (w * h * d);

}

Public class main {

Public static void main (String[] args) {

Box obj = new Box();

System.out.print (obj.volume());

(34) write a Program add & muliple of two number in use for class & object.

→ class math {

double n1 = 20;

double n2 = 30;

double add() {

return (n1 + n2);

double mul() {

return (n1 * n2);

}

Public class main {

Public static void main (String[] args) {

Math obj = new math();

System.out.println (obj.add());

System.out.println (obj.mul());

}

(35) Private access modifier

```
→ public class person {  
    private String name;  
    public void setName (String name) {  
        this.name = name;  
    }
```

```
    public String getName () {  
        return name; }  
}
```

```
public class main {
```

```
    public static void main (String [] args) {
```

```
        person p = new person();
```

```
        p.setName ("John");
```

```
        System.out.println ("Person name is : " + person  
                               .getName());  
    }
```

```
}
```

(36) Public Access modifier

```
→ public class Car {
```

```
    public String brand = "Toyota";
```

```
    public void displayBrand () {
```

```
        System.out.println ("Brand : " + brand); }  
}
```

```
public class main {
```

```
    public static void main (String [] args) {
```

```
        Car c = new Car();
```

```
        c.displayBrand ();  
    }
```

```
}
```


[37] Protected Access modify
→ class parent {
 protected String message = "Hello world";
}

```
public class child {  
    public static void main (String [] args) {  
        parent p = new parent ();  
        System.out.println (p.message);  
    }  
}
```

[38] Default Access modify
→ class Person {
 void ^{show} ~~main~~ () {

 System.out.println ("Hello world");
 }

```
public class default {  
    public static void main (String [] args) {  
        Person p = new Person ();  
        p.show ();  
    }  
}
```

[39] Using this to Refer to Instance variables.

```
→ class example {  
    int num;  
    example(int num) {  
        this.num = num;  
    }  
    void display() {  
        System.out.println("Value : " + this.num);  
    }  
    public static void main(String[] args) {  
        example obj = new example(10);  
    }  
}
```

[40] Using this to call Another Constructor.

```
→ class Example {  
    int num;  
    Example() {  
        this(100);  
        System.out.println("Default constructor");  
    }  
    Example(int num) {  
        this.num = num;  
        System.out.println("Parametrized " + num);  
    }  
    public static void main(String[] args) {  
        Example obj = new Example();  
    }  
}
```


[41]

Returning Current Class Instance

→

class Example {

int num;

Example.setNum(int num) {

this.num = num;

return this; }

void display () {

System.out.println("Number" + num); }

public static void main (String [] args) {

Example obj = new Example ();

obj.setNum(20).display ();

}

[42]

Using this keyword as a method Parameter in Java.

→

class Example {

void display () {

System.out.print("Display method"); }

void callMethod (Example obj) {

obj.display (); }

void paramthis () {

callMethod (this);

public static void main (String [] args) {

Example obj = new Example ();

obj.paramthis ();

}

}

[43] Using 'this' keyword to invoke the current class method.

```
→ class Ex {  
    void sh() {  
        System.out.println("Hello");  
    }  
    void di() {  
        System.out.println("world");  
        this.sh();  
    }  
}
```

```
public static void main (String args) {  
    Ex obj = new Ex();  
    obj.display();  
}
```


[44] A static variable is shared among all instance of a class.

```

→ class stu {
    int rollno;
    String name;
    static String clg = "ABC Uni";
    stu (int r, String n) {
        rollno = r;
        name = n;
    }
    void dis () {
        System.out.println(rollno + " " + name + " " + clg);
    }
}

public static void main (String args[]) {
    stu s1 = new stu (101, "Alice")
    stu s2 = new stu (102, "Bob")
    s1.display();
    s2.display();
}

```

[45] A static method can be called without creating an object.

```

→ class utility {
    static int cube (int x) {
        return x*x*x;
    }
}

public static void main (String[] args) {
    System.out.println (Utility.cube (3));
}

```

[46] A static block

```

→ class ex { static {
    System.out.println ("Hello");
}

    public static void main (String[] args) {
        System.out.println ("world")
    }
}

```

[47] A static nested class.

```

→ class Outer {
    static class Inner {
        void sh() {
            System.out.println ("static inner class")
        }
    }

    public static void main (String [] args) {
        Outer.Inner obj = new Outer.Inner();
        obj.sh();
    }
}

```

[48] Static import

```

→ import static java.lang.Math.*;

public class Test {
    public static void main (String [] args) {
        System.out.println (sqrt(25));
        System.out.println (pow(2,3));
    }
}

```


EXPT. NO. NAME ★ Overloading method [3 parameter]

[49] Different number of parameter.

```
→ class math {
    void add (int a, int b) {
        System.out.print("Sum: " (a+b));
    }
    void add (int a, int b, int c) {
        System.out.print("Sum: " (a+b+c));
    }
}
public static void main (String args) {
    math obj = new math();
    obj.add (10, 20)
    obj.add (10, 20, 30)
}
```

[50] Different data type of parameter.

```
→ class math {
    void mult (int a, int b) {
        System.out.println("Product(int): ", (a*b));
    }
    void mult (double a, double b) {
        System.out.println("Product(double): ", (a*b));
    }
}
public class test {
    public static void main (String args) {
        math obj = new math();
        obj.mult (5, 6)
        obj.mult (5.5, 6.5)
    }
}
```

EXPT. NO. NAME

[51] Overloading by changing the order of parameter.

```
→ class display {
    void show (String n, int num) {
        System.out.print("String & int : " + n + " " + num);
    }
    void show (int num, String n) {
        System.out.println("int & String : " + num + " " + n);
    }
}

public class test {
    public static void main (String [] args) {
        display obj = new display();
        obj.show ("Hello", 10);
        obj.show (10, "Hello");
    }
}
```

[52] final Variable

```
→ class Test {
    final int max = 100;
    void dis () {
        max = 200;
        System.out.println ("Max value : " + max);
    }
}

public static void main (String [] args) {
    Test obj = new test();
    obj.dis();
}
```

Teacher's Signature: _____

[53] Final method

```

→ class parent {
    final void show() {
        System.out.println("This is a final method"); } }
class child extends parent {
    void show() {
        System.out.println("Trying to override final method"); } }
public static void main (String[] args) {
    child obj = new child();
    obj.show();
}

```

[54] Final class [cannot be inherited]

```

→ final class vehicle {
    void dis () {
        System.out.println("Hello"); } }
public class test {
    public static void main (String[] args) {
        vehicle obj = new vehicle();
        obj.dis ();
    }
}

```

[55] final with Reference variables

```
→ class Test {
    int num = 10;
    public static void main (String [] args) {
        final test = 20;
        System.out.println (obj.num);
    }
}
```

[56] Single Inheritance

```
→ class A {
    void dis () {
        System.out.println ("Hello");
    }
}

class B extends A {
    void show () {
        System.out.println ("World");
    }
}

public static void main (String [] args) {
    B obj = new B();
    obj.show();
    obj.dis();
}
}
```


EXPT.
NO.

NAME

[57]

multilevel inheritance.

class A {

void dis () {

System.out.println ("I am");

}

class B extends A {

void show () {

System.out.println ("mishra");

}

class C extends B {

void vi () {

System.out.println ("Vishal");

}

public class main {

public class void main (String [] args) {

C obj = new C ();

obj.dis ();

obj.show ();

obj.vi ();

}

}

EXPT.
NO.

NAME

M T W T F S S

Page No.: 8

Date:

YOUVA

[58] Hierarchical Inheritance

```
class A {  
    void dis() {  
        System.out.println("Hello")  
    }  
}
```

```
class B extends A {  
    void sh() {  
        System.out.println("world");  
    }  
}
```

```
class C extends A {  
    void vi() {  
        System.out.print("Mej");  
    }  
}
```

```
public static void main (String [] args) {  
    C obj = new C();  
    B obj1 = new B();  
    obj.vi();  
    obj.dis();  
    obj1.sh();  
    obj1.dis();  
}  
}
```

Teacher's Signature:

[59] Multiple Inheritance [Using interface]

```

→ interface A {
    void dis();
}

interface B {
    void sh();
}

class C implement A, B {
    public void dis() {
        System.out.println("Hello");
    }

    public void sh() {
        System.out.println("world");
    }
}

public class main {
    public static void main (String [] args) {
        C obj = new C();
        obj.dis();
        obj.sh();
    }
}

```

NAME ★ Constructors

[60] Default Constructors

```

→ class A {
    String name;
    A() {
        name = "Hello";
    }
    void dis() {
        System.out.println(name);
    }
    public static void main (String [] args) {
        A obj = new A();
        obj.dis();
    }
}

```

[61] Parameterized Constructor

```

→ class A {
    String name;
    A (String n) {
        name = n;
    }
    void dis() {
        System.out.println(name);
    }
    public static void main (String [] args) {
        A obj = new A("Hello");
        obj.dis();
    }
}

```

Teacher's Signature: _____

[62] Copy constructors

→ class A {

String name;

A (String n) {

name = n; }

A (A s) {

name = s.name; }

void dis () {

System.out.println (name); }

public static void main (String [] args) {

A obj = new A ("Hello");

A obj3 = new A (obj);

obj3.dis ();

}}

[63] Passing object as a parameter.

→ class stu {

String name;

stu (String n) {

name = n;

void dis () {

System.out.println (name); }

void updatestu (stu s) {

this.name = s.name; }

public static void main (String [] args) {

stu s1 = new stu ("Vishal");

stu s2 = new stu ("Hi");

s1.updatestu (s2);

s1.dis ();

}}

EXPT. NO. NAME

Constructor Overloading

```

class student {
    String name, int num;
    student () {
        name = "Vishal";
    }
    student (String n) {
        name = n;
    }
    student (String n, int a) {
        name = n;
        num = a;
    }
    void display () {
        System.out.println (name + age);
    }
}

public static void main (String [] args) {
    student s1 = new student ("Vishal");
    student s2 = new student ("vishali");
    student s3 = new student ("Vish", 22);
    s1. display ();
    s2. display ();
    s3. display ();
}
    
```

Teacher's Signature: _____


```
(65) import Package
-> package himanshu;
   public class himanshu {
       public void h (int num) {
           int fact = 1;
           for (int i = 1; i <= num; i++)
               {
                   fact = fact * i;
               }
           System.out.println (fact);
       }
   }
```

```
package fact;
import himanshu.*;
```

```
Public class hi {
    public static void main (String args[])
    {
        himanshu obj = new himanshu ();
        obj.h (5);
    }
}
```

0

(66) Accessing variable using Super keyword.

```
class A {
    String name = "hi";
    void show() {
        System.out.print("Hello world");
    }
}
```

```
class B extends A {
    void hi() {
        System.out.print(super.name);
    }
}
```

```
public class main {
    public static void main (String[] args)
    {
        B obj = new B();
        obj.hi();
    }
}
```


(67) Accessing method using super keyword.

```
class A {
```

```
    void show ()
```

```
    {
```

```
        System.out.print ("Hello world");
```

```
    }
```

```
}
```

```
class B extends A {
```

```
    void hi ()
```

```
    {
```

```
        super.show();
```

```
    }
```

```
}
```

```
public class main {
```

```
    public static void main (String args[]) {
```

```
        B obj = new B();
```

```
        obj.hi();
```

```
    }
```

```
}
```

(68) Accessing constructor using super keyword.

```
→ class A {  
    String name;  
    A() {  
        name = "Vishal";  
    }  
}
```

```
class B extends A {  
    B() {  
        super();  
    }  
}
```

```
public class main {  
    public static void main (String[] args)  
    {  
        B obj = new B();  
    }  
}
```


[69] Abstract class

```
→ abstract class Animal {  
    abstract void sound();  
    void eat() {  
        System.out.println("This animal eats food");  
    }  
}
```

```
class Dog extends Animal {  
    void sound() {  
        System.out.print("Dog barks");  
    }  
}
```

```
class cat extends Animal {  
    void sound() {  
        System.out.print("meow");  
    }  
}
```

```
class harry extends Animal {  
    void sound() {  
        System.out.print("harry is here");  
    }  
}
```

```
public class main {  
    public static void main (String args) {  
        Dog D = new Dog();  
        Cat C = new Cat();  
        harry ha = new harry();  
    }  
}
```

```
D.Sound();
```

```
D.eat();
```

```
C.Sound();
```

```
C.eat();
```

```
ha.Sound();
```

```
ha.eat();
```

```
}
```

```
}
```

[70] One Try One Catch

→ Public class Trys {

```
    Public static void main (String [] args) {
```

```
        try {
```

```
            int a=10, b=0;
```

```
            int c = a/b;
```

```
        }
```

```
        catch (ArithmeticException e)
```

```
        {
```

```
            System.out.print ("Zero is divide");
```

```
        }
```

```
    }
```

```
}
```


[71] One by multiple catch

```
→ public class try {  
    public static void main (String[] args) {  
        try {  
            int b[] = {10, 20, 30};  
            System.out.println(b[5]);  
            int a = 10/0;  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("error");  
        }  
        catch (Exception g) {  
            System.out.println("wrong")  
        }  
    }  
}
```

(72) Nested try catch

```
→ public class nested {  
    public static void main (String [] args) {  
        try {  
            int [] number = {5, 10, 15};  
            System.out.println ("Access ", number [5]);  
            try {  
                int result = number [0] / 0;  
                System.out.println ("Result " + result);  
            }  
            catch (ArithmeticException e)  
            {  
                System.out.print ("Cannot divide by zero");  
            }  
        }  
        catch (ArrayIndexOutOfBoundsException o)  
        {  
            System.out.println ("Error in array");  
        }  
    }  
}
```


throws Statement

```
public class EasyThrowsExample {  
  
    public static void divide(int a, int b) throws ArithmeticException {  
        System.out.println("Result: " + (a / b));  
    }  
  
    public static void main(String[] args) {  
        try {  
            divide(10, 0); // This will cause an exception (division by zero)  
        } catch (ArithmeticException e) {  
            System.out.println("Error: Cannot divide by zero!");  
        }  
    }  
}
```

Source Refactor Run Debug Profile Team Tools Window Help

Search (Ctrl+I)

<default confi... 249.7/640.0MB

...va Uzair.java × Exam.java × array.java × nestedtry.java × TryCatch.java × Man.java ×

```
15 public class Man {
16     public class Man {
17         public static void readfile(String filename) throws IOException{
18             FileReader file = new FileReader(filename);
19             BufferedReader fileInput = new BufferedReader(file);
20             System.out.println(fileInput.readLine());
21             fileInput.close();
22         }
23     }
24     public static void main(String[] args) {
25         try{
26             readfile("/home/student/Desktop/mann.txt");
27         }catch (Exception e){
28             System.out.println("An error : "+e);
29         }
30     }
31 }
32
33
34
```

(run)

LD SUCCESSFUL (total time: 0 seconds)

Output

(1) 33:2 INS